GenAI in the Enterprise: Knowledge Architecture Before the Model

About me

Contacts

Why this topic

What you'll get from this talk:

Data vs Knowledge (why it matters)

What “Knowledge Architecture” is

How MongoDB fits as a structural layer

A quick demo (metadata-first retrieval)

The GenAI gold rush

“LLM + data + RAG” looks great in demos

Many teams ship a chatbot fast

Scaling pain

What breaks in real companies

- Inconsistent answers
- Conflicting documents
- No traceability (“where did this come from?”)
- Outdated content / multiple versions

The wrong question

It’s not “Which model?”

- Wrong: “Which LLM should we pick?”
- Right: “How is our knowledge structured and governed?”

Data is not knowledge

The key distinction

- Data: files, PDFs, tables, dumps
- Knowledge: context, structure, relationships, lifecycle

What is Knowledge Architecture

- **Right granularity (not too big, not too tiny)**
- **Rich metadata**
- **Versioning & lifecycle (active/deprecated)**
- **Separation of content vs context**
- **Auditability & governance**

A common anti-pattern (quick)

“Dump everything into a vector store”

- Seems to work at first
- Fails with ambiguity, weak context, and maintenance issues
- Sets up the demo (without going deep into RAG mechanics)

  10k PDFs. Let’s chunk them, embed them, and search vectors.
  Same topic exists in multiple versions (v2, v3).
  Same policy differs by region (EU vs US).
  Without structure, retrieval returns “relevant” but not “correct”.

Where MongoDB fits

MongoDB as a structural knowledge layer

- Not just storage
- Source of truth: documents + metadata + lifecycle
- Helps retrieve the *right* context before the model

A minimal example model

What “AI-ready knowledge” looks like

- One collection: `knowledge_items`
- Fields: `doc_type`, `domain`, `region`, `status`, `version`, `effective_date`, `tags`, `body`

  This is the smallest useful unit of knowledge.”
  The LLM doesn’t need the whole PDF. It needs the right policy, the right version, for the right region.

  Metadata isn’t bureaucracy. It’s how you prevent wrong context.

- Message: structure first, embeddings second

Metadata-first retrieval - Demo

Naive search: “password policy” returns mixed versions

Structured retrieval

Optional: paste the correct excerpt into an LLM prompt (manual, low risk)

6 docs in `knowledge_items`:

1. Password Policy v2 (deprecated, EU)
2. Password Policy v3 (active, EU)
3. Onboarding checklist (EU)
4. Onboarding checklist (US)
5. VPN access procedure (EU)
6. FAQ: Password reset (EU)

- Insert into MongoDB `knowledge_items` with metadata: region, status, version, doc_type.
- Build **Naive retrieval**: search only by text/similarity (no metadata filters) → returns v2 + v3.
- Build **Structured retrieval**: filter `region=EU`, `status=active`, `doc_type=policy` + pick latest version → returns v3.
- Use the same Gemini prompt for both runs

Demo:
Q: "*What is our password policy for the EU?*"
Naive result: include V2 and V3
Struct retrieval: filter `status=active`, `region=EU`, sort by `version → result is just V3`

What changes in practice

Why this matters

- More consistent answers
- Less “hallucination from weak context”
- Easier auditing and debugging
- Easier updates and ownership

Thank you
Models change. Architecture stays.

Q&A
Linkedin